Les bases de la programmation en Python

Objectifs

1	Premiers pas en Python	Découvrir les structures de données de Python. Définir les caractéristiques de chaque structure de données. Comprendre comment manipuler des listes, des dictionnaires, des tuples et des sets.
2	Syntaxe de base de Python	Apprendre la syntaxe de base de Python. Explorer l'indentation et les commentaires en Python. Comprendre comment importer des bibliothèques de Python.
3	Les bases de Python	Découvrir les différents types de données de Python (int / float / string / boolean). Manipuler les opérations de base de Python en utilisant les opérateurs arithmétiques et logiques...). Créer des structures conditionnelles avec Python. Utiliser les boucles (“for” et “while”).
4	Structures de données de Python	Découvrir les structures de données de Python. Définir les caractéristiques de chaque structure de données. Comprendre comment manipuler des listes, des dictionnaires, des tuples et des sets.

Objectifs

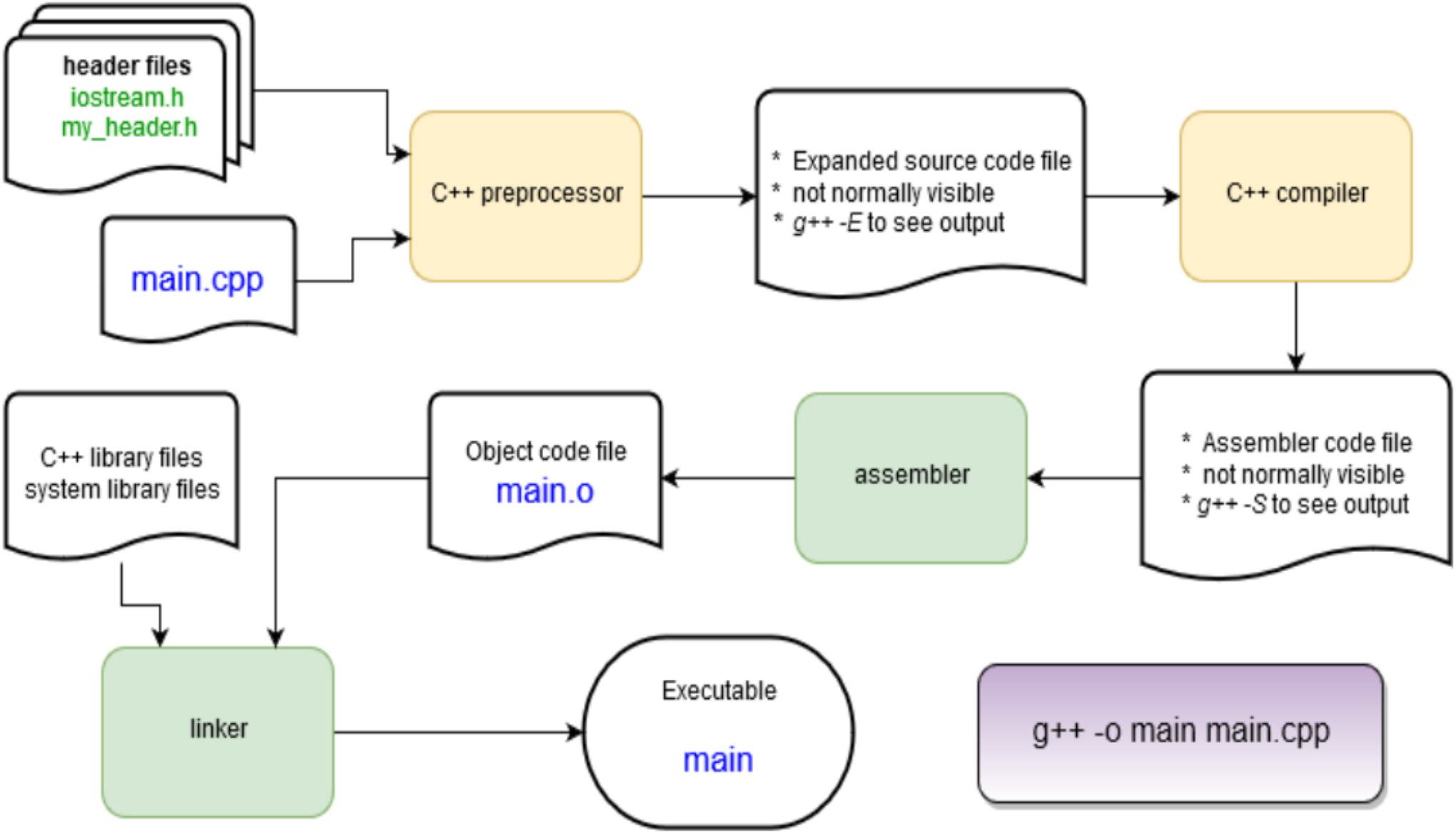
5	**Les fonctions de Python**	Définir des fonctions en Python. Comprendre la fonction Lambda en définissant son objectif et sa structure. Expliquer l'utilisation des paramètres et des arguments dans la construction de fonctions Python.
6	**Classes et objets avec Python**	Définir des objets et des classes en Python. Faire la différence entre objet et classe. Expliquer comment manipuler des objets et des classes avec Python.
7	**La bibliothèque Numpy**	Définir et créer des tableaux en Python. Découvrir et utiliser la bibliothèque Numpy. Expliquer les principales opérations utilisées dans les tableaux Numpy. Manipuler quelques fonctions usuelles associées à Numpy en Python.
8	**La bibliothèque Pandas**	Découvrir la bibliothèque Pandas. Découvrir comment construire des dataframes. Apprendre à manipuler les “dataframes” et les séries en Python.

Overview

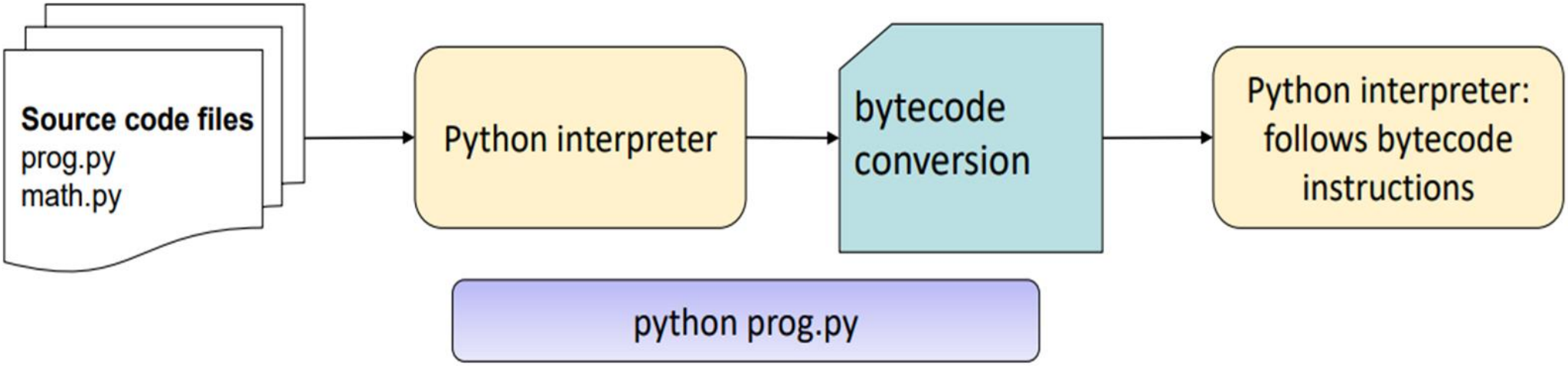
Python ?

- **Easy** and **intuitive language** just **as powerful as major competitors**
- **Open source.**
- **Understandable.**
- allowing **short development times.**
- Is a **general purpose interpreted programming language.**
- Is a language that supports multiple approaches to software design, Principally **structured** and **object-oriented** programming.
- Provides **automatic memory management** and **garbage collection.**
- Is **extensible.**
- Is **dynamically typed.**

Compiled languages (ex. C, C++, Fortran, …)



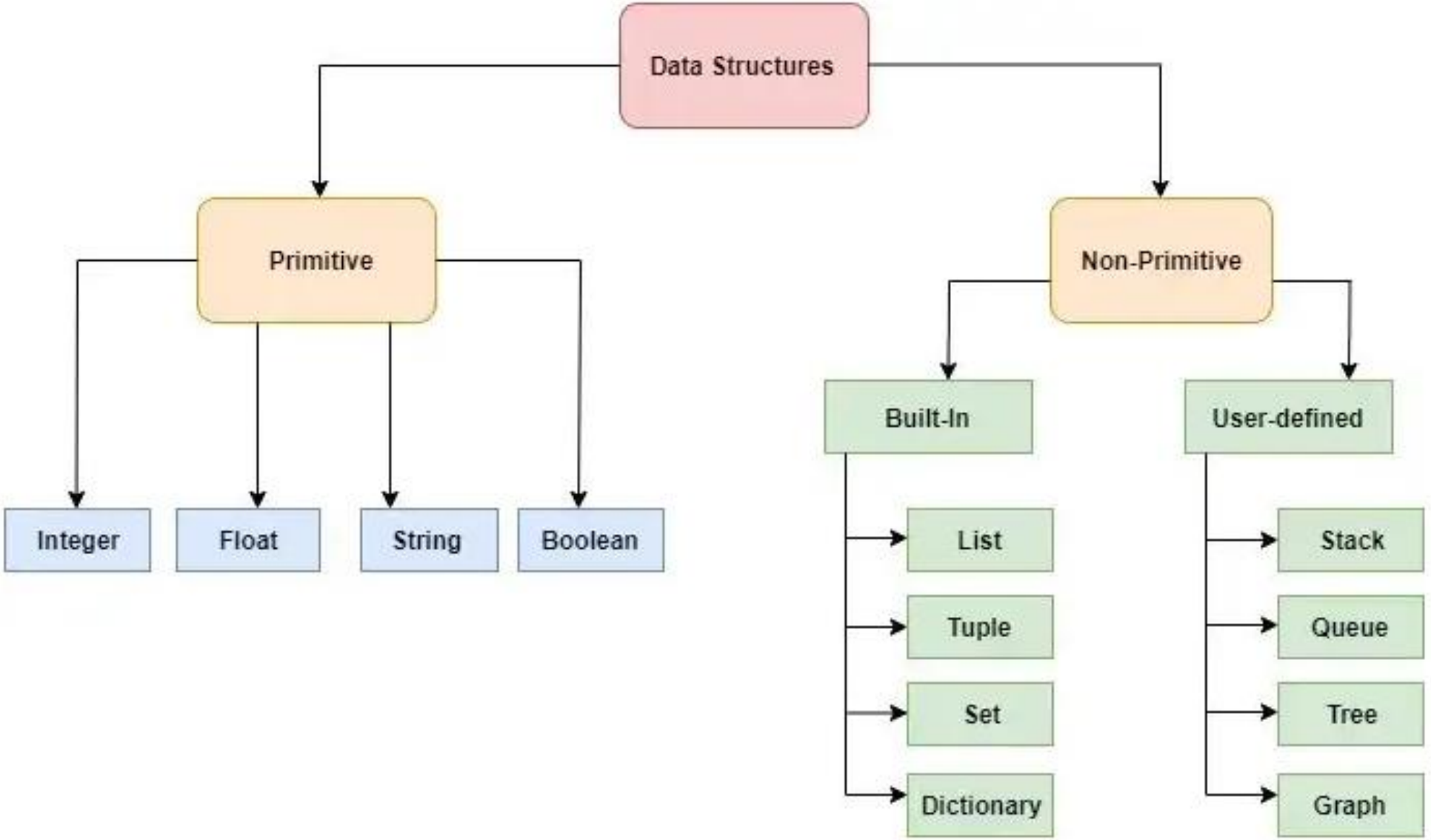
Python is an interpreted language



- **Faster development: A lot less work to get a program Python to start running** compared with compiled languages!
- Bytecodes are an internal representation of the text program that can **be efficiently run by the Python interpreter**.
- The interpreter itself is written in C and is a compiled program.
- **Slower programs**

Python standard Data types

Python provides various **standard data types** that define the storage method on each of them. The data types defined in Python are given below:



Python Operators

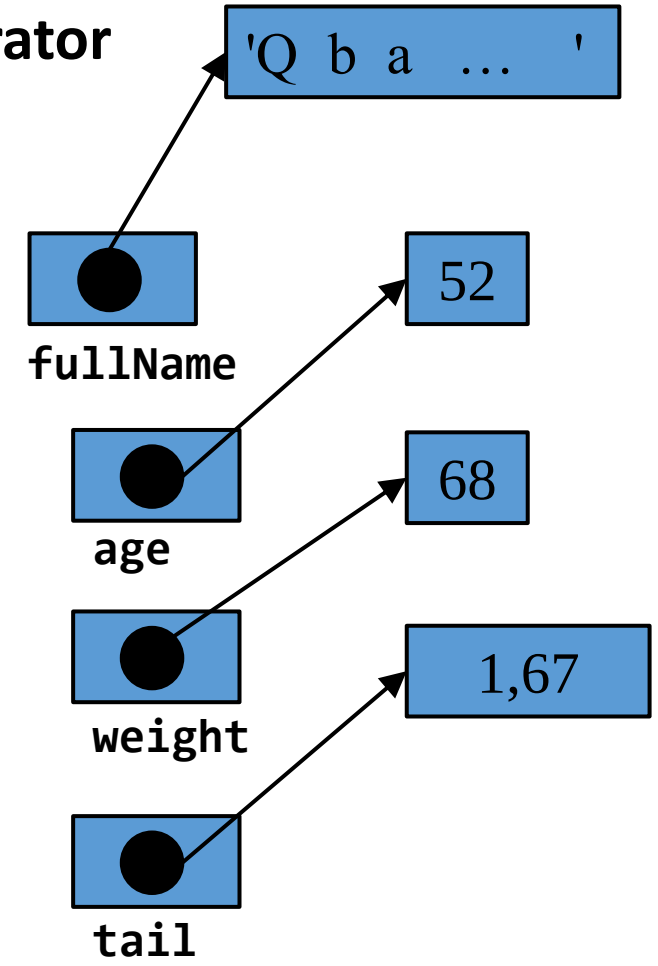
Python supports a **wide variety of operators** :

- **Arithmetic:** + - * / // % **
- **Comparison:** > < >= <= != ==
- **Logical:** and or not
- **Bitwise:** & | ~ ^ >> <<
- **Assignment:** = += -= *= /= //= %= **== &=
  |= ~= ^= >>= <<=
- **Identity:** is is not
- **Membership:** in not in
- **Operator Precedence: ()**

Variables

- Variables are **assigned values using the = operator**
- Variables **can be reassigned** at any time
- Variable **type is not specified**
- **Types can be changed with a reassignment**

```
fullName = 'Qbadou Mohammed'
age  = 52
weight = 1.67
tail = 68
weight = 68
tail = 1.67
```



Variable names:

- Must begin with a letter (a - z, A - B) or underscore _
- Other characters can be letters, numbers or _
- Are case sensitive: capitalization counts!
- Can be any reasonable length

Sommaire

1. Lists

- Objective
- Motivation
- Construction
- Indexing/Slicing operators
- Built-in methods
- Use cases

Objective

- How to Manipulate Collections of Values
- How to Write Python Lists
- How to browse a Python list or access one of its items

Motivation

If we want to average three numbers, we can easily do the following:

```
num1 = int(input('Please enter a number: '))
num2 = int(input('Please enter a number: '))
num3 = int(input('Please enter a number: '))

print((num1 + num2 + num3) / 3)
```

What if we want to average a 1000 different numbers? Do we make a 1000 different variables?

The answer is of course no as this will result in **overly long, non-generic and impractical write-ups**

Lists

List operations

Lists are a way of **storing multiple pieces of information** in the same place! Python list operations: creating lists, changing list elements, removing elements, and other list operations

1. Creating lists

```
my_list = [9, 10, 11, 17, 21] #list of three integers
list1 = ['Ali','Mohamed','Fatima','Zineb'] #list of Strings
list2 = [ ] #create an empty list
list3 = list() #create an empty list
list4 = [[4,3,2],[3,5,7]] #Nested list : list of lists
list5 = ['Ali',9,11, [4,3,2]] #list of heterogeneous items
```

2. Indexing/slicing /changing list items

```
print(my_list[0]) # print the first item
print(my_list[1:3]) #prints items with indices from 1 to 2.
my_list[0]=2*my_list[1] # => [20, 10, 11, 17, 21]
print(list4[0][2]) # Nested indexing => 2
```

List operations

3. Indexing/slicing /changing list items

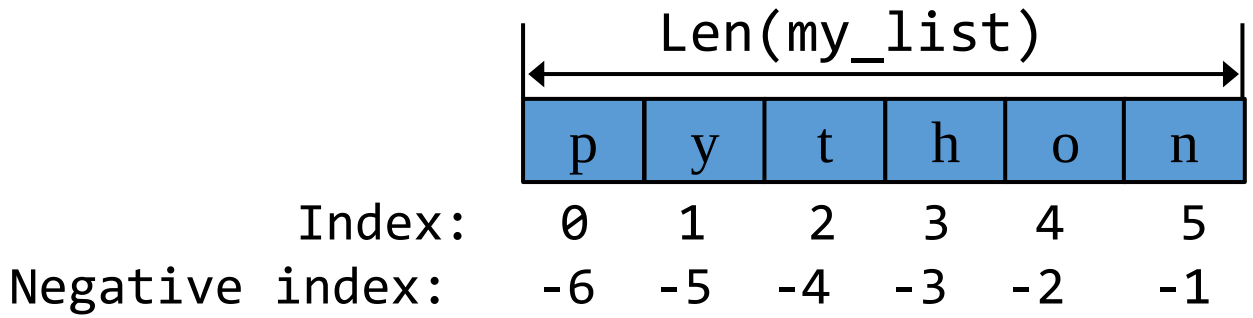
We can refer to items in the list by their number (or index). The index can be an number or a variable or an arithmetic expression.

```
i = 5
print(my_list[2],my_list[i],my_list[2*i])
```

4. Negative indexing

Python allows **negative indexing** for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

```
my_list = ['p','y','t','h','o','n']
```




```
print(my_list[-1]) # last item => n
print(my_list[-5]) # fifth last item => y
print(my_list[-5:-2])#sublist from -5 to -3=>['y','t',’h’]]
```

List built-in methods

In addition to **indexing** and **slicing** operations on a list, Python has a set of built-in methods that we can use on lists:

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Append method

The append() method appends an element to the end of the list :

Syntax

```
my_list.append(new_item)
```

Examples

```
my_list = [ 3, 4, 5]
my_list.append(6)
print(my_list)          # => [3, 4, 5, 6]
my_list.append([9, 11]) # appends [9, 11] as the last item
print(my_list) #=> [3, 4, 5, 6, [9, 11]]
fruits = ["apple", "banana", "strawberry", "grape"]
vehicle_brands = ["Ford", "BMW"]
fruits.append(vehicle_brands)
print(fruits) #=> ["apple", "banana", "strawberry",
                   #"grape", ["Ford", "BMW"]]
```

Remove by index - pop method

Syntax

To delete something at a certain **index**, just call:

pop(index)

Example

```
myList = [10, 12, 14]

myList.pop(1)

print(myList) # => [10, 14]
```

Remove by value – Remove method

Syntax

To delete a certain value from a list, use:

remove(value)

Example

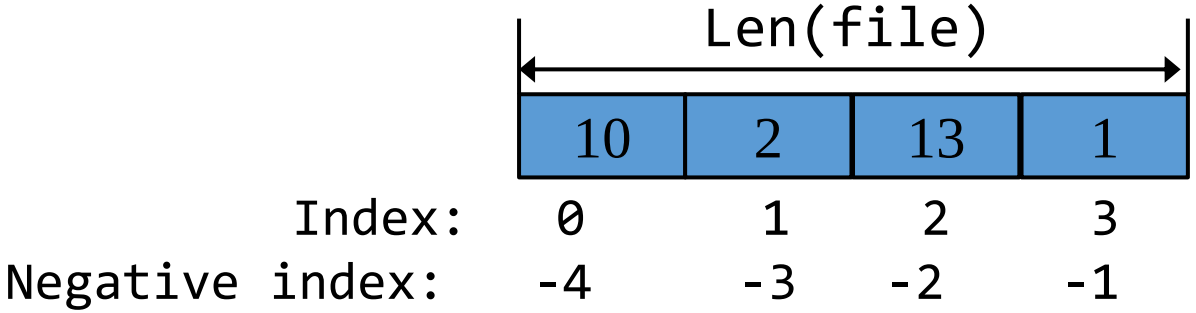
```
myList = [10, 12, 14, 15, 14]

myList.remove(14)

print(myList) # => [10, 12, 15, 14]
```

Manipulation d'une Liste d'entiers

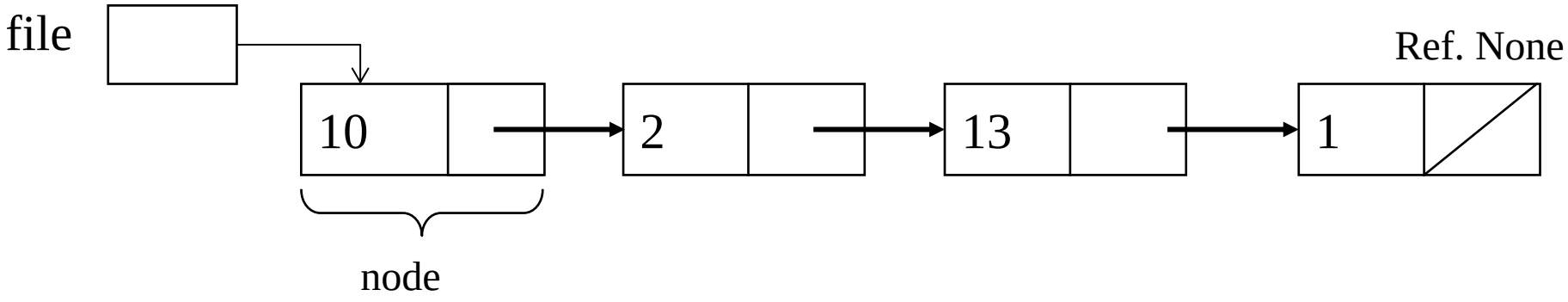
Ecrire une application python qui gère une liste d'entiers qui représente le comportement de fille d'attente (FIFO) par les opérations suivantes :

- Enfiler un élément => Insertion en queue de la liste
- Défiler un élément si la file n'est pas vide => Suppression de l'élément en tête
- Consulter l'élément en tête de la file si la file n'est pas vide
- Verifier si la liste est vide

Première implémentation - utilisant python list



Première implémentation - utilisant python list



Use case 2

Let, X is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression Y.

1.Push "(" onto Stack, and add ")" to the end of X.
2.Scan X from left to right and repeat Step 3 to 6 for each element of X until the Stack is empty.
3.If an operand is encountered, add it to Y.
4.If a left parenthesis is encountered, push it onto Stack.
5.If an operator is encountered ,then:
    1. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) which has the same precedence as or higher precedence than operator.
    2. Add operator to Stack.
    [End of If]
6.If a right parenthesis is encountered ,then:
    1. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) until a left parenthesis is encountered.
    2. Remove the left Parenthesis.
    [End of If]
    [End of If]
7.END.